

PE-EC703B — Wireless Sensor Networks

Semester VII • 3L:0T:0P • 3 Credits

Module 5 Exam Notes: Design Principles, Gateway Concepts & TinyOS/nesc

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Design Principles for Wireless Sensor Networks	2
2	Gateway Concepts in WSN	3
2.1	Need for a Gateway	3
2.2	Gateway Architecture and Functions	3
3	WSN to Internet Communication	4
3.1	Integration Approaches	4
3.2	6LoWPAN (IPv6 over Low-Power Wireless PANs)	4
3.3	CoAP (Constrained Application Protocol)	4
4	Internet to WSN Communication	5
4.1	Challenges of Internet-to-WSN Downlink	5
4.2	Mechanisms	5
5	Operating Systems for WSN	6
5.1	Comparison: WSN OS vs General-Purpose OS	6
6	TinyOS	6
6.1	Key Features of TinyOS	6
7	nesc Programming Language	7
7.1	Key nesc Concepts	7
7.2	TinyOS vs Linux vs RTOS	8

1. Design Principles for Wireless Sensor Networks

WSN design is fundamentally different from traditional network design. The scarcity of energy, computation, and memory requires application-aware, cross-layer, and energy-first design principles.

Core WSN Design Principles

1. **Energy efficiency first:** Every design decision — protocol, algorithm, hardware — must minimize energy consumption. Network lifetime is the primary performance metric. Aggressive duty cycling (radio sleep), in-network processing, and shortest-path routing are non-negotiable.
2. **In-network processing (data-centric design):** Perform aggregation, filtering, and fusion inside the network, not just at the sink. A cluster head that averages 10 readings saves $9\times$ the transmission energy of forwarding raw data.
3. **Cross-layer design:** Traditional layered (OSI) design isolates layers. WSN protocols jointly optimize MAC + routing + application (e.g., LEACH combines clustering at the network layer with TDMA at the MAC layer, driven by application data rate).
4. **Self-organization and adaptivity:** The network must configure itself, elect cluster heads, discover routes, and heal from failures without human intervention. Protocols must work correctly under random deployment and topology changes.
5. **Collaboration and redundancy:** Exploit spatial redundancy — multiple nodes cover the same area. Use data from multiple sensors for reliability and accuracy (spatial diversity). Active nodes can cover for sleeping or dead nodes.
6. **Data-centric communication:** Address data by its attributes (“temperature > 40°C”), not by node IDs. This enables in-network aggregation, suppression of duplicate data, and semantic routing.
7. **Quality/fidelity trade-off:** Accept lower resolution or less frequent data in exchange for energy savings. Adaptive sampling reduces data rate when environment is stable, increases it during events.
8. **Security by design:** Assume a hostile deployment environment. Build authentication and freshness into every protocol (not bolted on after). Use lightweight symmetric cryptography (AES-128) rather than public-key.

2. Gateway Concepts in WSN

WSN Gateway — Definition

A **WSN gateway** is a device that bridges the wireless sensor network with an external network (such as the Internet, an enterprise LAN, or a cellular network). It performs **protocol translation** between the low-power WSN protocols (IEEE 802.15.4 / ZigBee) and standard IP networking (TCP/IP / HTTP).

2.1 Need for a Gateway

- WSN nodes speak IEEE 802.15.4 / ZigBee; the internet speaks TCP/IP. These are incompatible at multiple layers.
- Sensor nodes are too resource-constrained to run a full TCP/IP stack.
- The gateway provides a management interface for the WSN (task injection, firmware update, health monitoring).
- The gateway buffers data from the WSN and forwards to cloud/server in bursts, reducing WSN active time.

2.2 Gateway Architecture and Functions

WSN Gateway Integration Diagram



Gateway functions:

- **Protocol translation:** Converts ZigBee/IEEE 802.15.4 frames to IP packets and vice versa.
- **Data buffering:** Stores sensor data locally (flash/SD card) if internet is unavailable.
- **Data formatting:** Converts raw sensor values to JSON, XML, or MQTT payloads for cloud APIs.
- **Security bridging:** Translates between WSN security (AES-128 with SNEP) and internet security (TLS/SSL).
- **Task injection:** Receives queries from the internet and injects them as interests into the WSN.
- **Network management:** Monitors WSN health, tracks node battery levels, and triggers alarms.

3. WSN to Internet Communication

WSN-to-Internet Communication

WSN to Internet communication refers to the uplink flow where sensor data is transported from the sensor field, through the gateway, to an Internet-connected server or cloud platform for storage, processing, and visualization.

3.1 Integration Approaches

Approach	Description	Protocols
Gateway-based (Overlay)	WSN and Internet are two separate networks; gateway translates between them	ZigBee + HTTP/MQTT gateway
6LoWPAN (IP-native)	Runs IPv6 directly on IEEE 802.15.4 nodes using header compression; no separate gateway needed	6LoWPAN + CoAP + RPL
Middleware approach	Software layer abstracts WSN heterogeneity; presents a uniform API to internet applications	REST-based middleware

3.2 6LoWPAN (IPv6 over Low-Power Wireless PANs)

6LoWPAN — Definition

6LoWPAN is an IETF adaptation layer (RFC 4944) that enables **IPv6 packets to be transmitted over IEEE 802.15.4** networks by compressing IPv6 headers (from 40 bytes to 2–6 bytes) and fragmenting large packets to fit the 127-byte IEEE 802.15.4 frame. This allows sensor nodes to participate directly in the Internet of Things.

Key functions of 6LoWPAN adaptation layer:

- **Header compression:** IPv6 header (40 B) + UDP (8 B) compressed to ≤ 6 B using IPHC (IP Header Compression).
- **Fragmentation and reassembly:** IPv6 MTU = 1280 B; IEEE 802.15.4 payload = 102 B (max). 6LoWPAN fragments and reassembles.
- **Mesh addressing:** Supports multi-hop routing for IEEE 802.15.4 using mesh header.
- **Neighbor discovery:** Optimized ND for low-power, lossy networks (ND-LowPAN).

3.3 CoAP (Constrained Application Protocol)

CoAP — Definition

CoAP is a lightweight application protocol designed for constrained IoT and WSN devices, functioning as the “HTTP for IoT.” It runs over UDP (instead of TCP), uses a REST-based request/response model, and supports GET, PUT, POST, DELETE operations on resources (sensor readings, actuator states).

- **Binary encoding:** Compact binary header (4 bytes fixed header vs HTTP's 100s of bytes).
- **UDP-based:** Avoids TCP's connection overhead and handshake energy cost.
- **Confirmable messages:** CoAP has its own reliability mechanism (CON messages with ACK).
- **Observe extension:** Server pushes updates to subscribed clients (no polling needed).
- **Gateway translation:** A CoAP-HTTP proxy converts between CoAP (sensor side) and HTTP (internet side).

4. Internet to WSN Communication

Internet-to-WSN Communication

Internet to WSN communication refers to the downlink flow: queries, commands, firmware updates, and configuration sent from an Internet-connected server or user application into the sensor network.

4.1 Challenges of Internet-to-WSN Downlink

- **Sleeping nodes:** Sensor nodes may be asleep when a command arrives. The gateway must buffer the command and deliver it at the node's next wake-up.
- **Asymmetric bandwidth:** Internet downlink can push data faster than the WSN can absorb.
- **Addressing:** Individual nodes may have only 16-bit short addresses (not globally routable). The gateway maps external requests to internal WSN addresses.
- **Reliability:** IEEE 802.15.4 is lossy; downlink commands may need retransmission.

4.2 Mechanisms

- **MQTT (Message Queuing Telemetry Transport):** Publish-subscribe protocol over TCP; the broker buffers messages for offline sensor nodes. Gateway subscribes to topics and injects commands into WSN.
- **RPL (Routing Protocol for Low-Power and Lossy Networks):** IETF routing protocol for 6LoWPAN; builds a Destination-Oriented DAG (DODAG) from the sink downward, enabling Internet-initiated routes.
- **CoAP Observe:** Internet server registers observation on a sensor resource; sensor pushes updates. Reverse: server PUTs commands to sensor actuator resources.

5. Operating Systems for WSN

WSN Operating System — Requirements

An **embedded OS for WSN** must: fit in < 10 KB RAM and < 100 KB Flash, support concurrent tasks (sensing, routing, communication) on a single MCU, manage radio duty cycling, provide hardware abstraction for different sensor platforms, and be energy-aware (support for sleep modes).

5.1 Comparison: WSN OS vs General-Purpose OS

Feature	General-Purpose OS (Linux)	WSN OS (TinyOS)
RAM footprint	10s of MB	2–10 KB
Flash/ROM	100s of MB	30–100 KB
Scheduling model	Preemptive multi-threading	Event-driven (non-preemptive)
Energy management	Limited	First-class: radio, CPU sleep control
Hardware abstraction	Device drivers, system calls	Component interfaces (nesC)
Process isolation	Yes (MMU)	No MMU; single address space
Concurrency	Full multi-process/thread	Events + tasks (split-phase)

6. TinyOS

TinyOS — Definition

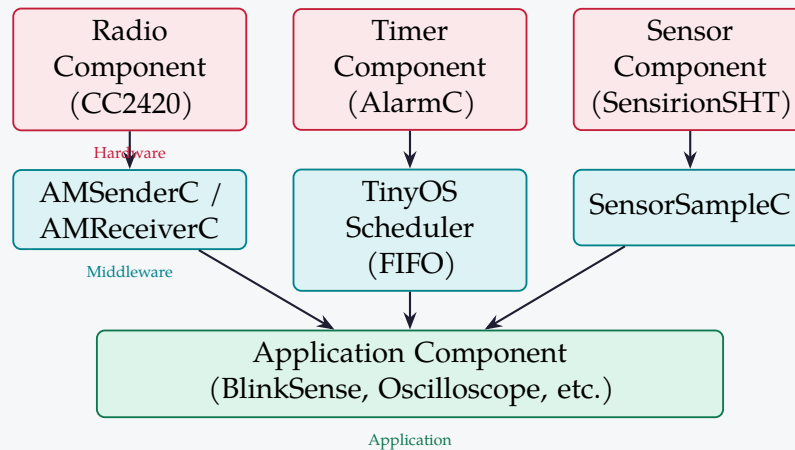
TinyOS is an open-source, **event-driven** operating system designed for resource-constrained sensor network nodes. Developed at UC Berkeley (2000), it is implemented in **nesC** and uses a **component-based architecture** where applications are assembled from reusable hardware and software components.

6.1 Key Features of TinyOS

- **Event-driven model:** No blocking calls. All long-running operations are **split-phase**: operation is started with a *command*, and its completion triggers an *event*.
- **Small footprint:** Core TinyOS < 400 bytes. Entire system with a basic radio stack fits in < 15 KB Flash.
- **Component-based:** Applications are composed of **components** that communicate through **interfaces**. No dynamic linking.
- **Two-level concurrency:** **Events** (hardware interrupts, split-phase completions) run at high priority; **Tasks** (deferred computation) run in FIFO order at lower priority.

- **Compile-time configuration:** Component wiring is done at compile time (by the developer using a “wiring” file), not at runtime. This removes overhead of runtime dynamic dispatch.
- **TinyOS scheduler:** Single FIFO task queue. Events can interrupt tasks. Tasks run to completion (non-preemptive).

TinyOS Component Model



7. nesC Programming Language

nesC — Definition

nesC (network embedded systems C) is a component-based, event-driven extension of the C programming language designed specifically for TinyOS and sensor network applications. It provides the **component model**, **interface** declarations, and **wiring** syntax to compose TinyOS applications from reusable modules.

7.1 Key nesC Concepts

1. Interfaces

An **interface** defines a set of bidirectional operations:

- **Commands:** Called by the user of the interface (top-down call). E.g., `AMSend.send()`.
- **Events:** Signaled by the provider of the interface (bottom-up notification). E.g., `AMSend.sendDone()`.

2. Components

Two types of components:

- **Module:** Implements an interface; contains C-like code for commands and event handlers.
- **Configuration:** Wires components together (no code, only connections).

3. Split-Phase Operations

Long-running operations (radio TX, ADC conversion) are split into:

- **Initiation:** Call a command (e.g., `call AMSend.send(...)`) — returns immediately.
- **Completion:** An event is signaled when done (e.g., `event AMSend.sendDone(...)`). No blocking.

4. Tasks

- Deferred computation: `post myTask();` schedules a task in the FIFO queue.
- Tasks run to completion; they cannot be preempted by other tasks (only by events/interrupts).

5. Atomicity and Race Conditions

- `atomic { ... }` block: disables interrupts for short, critical sections.
- nesC compiler performs a **race condition analysis** at compile time to flag potential data races between tasks and events.

7.2 TinyOS vs Linux vs RTOS

Feature	TinyOS	Linux	RTOS (FreeRTOS)
RAM	2–10 KB	10s MB	5–50 KB
Scheduling	Event-driven	Preemptive	Preemptive
Tasks	FIFO, non-preemptive	Preemptable threads	Priority-based tasks
Language	nesC (C extension)	C	C
Energy mgmt	First-class	Limited	Good
Blocking calls	Not allowed	Allowed	Allowed
Use case	WSN motes (8/16-bit MCU)	General-purpose (Raspberry Pi)	IoT/embedded (ARM Cortex-M)

Quick Revision — Module 5

Design Principles, Gateway Concepts & TinyOS/nescC | PE-EC703B

Key Concepts at a Glance

6LoWPAN Compression	IPv6 (40 B) + UDP (8 B) → IPHC compressed header (≤ 6 B)
IEEE 802.15.4 Payload	Max payload: 127 B (PHY layer), 102 B (MAC layer after headers)
IPv6 MTU size	1280 B ($\Rightarrow$ 6LoWPAN fragmentation required for WPAN link)
CoAP Fixed Header	Only 4 bytes (vs HTTP which requires > 100 bytes header)
TinyOS Scheduler	FIFO task queue; non-preemptive tasks; events interrupt tasks
nescC Split-Phase	call <code>Iface.start()</code> initiates; event <code>Iface.startDone()</code> signals completion
Gateway Function	Protocol translation (ZigBee $\leftrightarrow$ TCP/IP), buffering, security bridging

One-Line Definitions

- **WSN Gateway:** Bridges IEEE 802.15.4/ZigBee WSN to Internet via protocol translation and data formatting.
- **6LoWPAN:** IETF adaptation layer running IPv6 over IEEE 802.15.4 via header compression and fragmentation.
- **CoAP:** Lightweight REST protocol over UDP for constrained IoT devices; the “HTTP of IoT.”
- **RPL:** IETF routing protocol for 6LoWPAN; builds a DODAG (Destination-Oriented DAG) for uplink and downlink routing.
- **MQTT:** Publish-subscribe message protocol over TCP; broker buffers messages for offline sensor nodes.
- **TinyOS:** Event-driven, component-based OS for sensor motes; <15 KB footprint; non-preemptive.
- **nescC:** System programming language for TinyOS; extends C with components, interfaces, and wiring.
- **Split-phase:** Operation model where a command initiates and an event signals completion; no blocking.

Frequently Asked / Viva Questions

Q1. What are the key design principles for WSN?

Eight principles: (1) Energy efficiency first; (2) In-network processing/aggregation; (3) Cross-layer design; (4) Self-organization and adaptivity; (5) Collaboration and redundancy; (6) Data-centric communication; (7) Quality/fidelity trade-off; (8) Security by design. Energy is always the primary constraint driving all design decisions.

Q2. What is a WSN gateway? What are its functions?

A WSN gateway bridges the sensor network (IEEE 802.15.4/ZigBee) to the Internet. Functions: (1) Protocol translation (ZigBee $\leftrightarrow$ TCP/IP); (2) Data buffering; (3) Data formatting (JSON/MQTT); (4) Security bridging (WSN AES $\leftrightarrow$ internet TLS); (5) Task/query injection into WSN; (6) Network management and monitoring.

Q3. What is 6LoWPAN? Why is it needed?

6LoWPAN (IPv6 over Low-Power WPANs) is an adaptation layer (RFC 4944) enabling IPv6 packets to run over IEEE 802.15.4. It is needed because: IPv6 header = 40 B but 802.15.4 payload = 102 B (only 62 B left for UDP+data after raw IPv6). 6LoWPAN compresses IPv6+UDP to ≤ 6 B using IPHC and fragments large packets, enabling IP-native WSN nodes.

Q4. What is CoAP? How does it differ from HTTP?

CoAP (Constrained Application Protocol) is a lightweight REST protocol for IoT running over UDP. Differences from HTTP: runs over UDP (not TCP), has only 4-byte fixed header (vs 100+ bytes in HTTP), uses binary encoding (not text), includes built-in reliability (CON/ACK), supports observe mode (server push). Designed for < 10 KB RAM devices.

Q5. Explain TinyOS architecture.

TinyOS is an event-driven, component-based OS. Architecture: (1) **Components** — modules (implement code) and configurations (wire components); (2) **Interfaces** — define commands (calls) and events (callbacks); (3) **Scheduler** — FIFO task queue; tasks run to completion; events can interrupt tasks; (4) **Split-phase** — no blocking calls; operations initiated by commands, completed by events. Footprint: < 15 KB Flash, < 4 KB RAM.

Q6. What is nesC? Explain its key concepts.

nesC (network embedded systems C) is a component-based C extension for TinyOS. Key concepts: (1) **Interface** — defines commands (top-down) and events (bottom-up); (2) **Module** — implements an interface; (3) **Configuration** — wires components at compile time; (4) **Split-phase** — non-blocking; (5) **Tasks** — deferred computation posted to FIFO queue; (6) **Atomic** — interrupt-disable for race-condition-free critical sections.

Q7. What is a split-phase operation in TinyOS? Give an example.

A split-phase operation separates initiation from completion: the call initiates an operation and returns immediately (no blocking); an event fires when the

Frequently Asked / Viva Questions (contd.)

operation is done. Example: `call AMSend.send(msg)` starts radio transmission and returns immediately. When TX is done, event `AMSend.sendDone(msg, error)` is triggered for the application to handle success/failure.

Q8. Compare TinyOS, Linux, and FreeRTOS for WSN use.

TinyOS: 2–10 KB RAM, event-driven, FIFO non-preemptive tasks, designed for 8/16-bit WSN motes. Linux: 10s MB RAM, preemptive threads, general-purpose, too heavy for motes. FreeRTOS: 5–50 KB RAM, preemptive priority tasks, good energy management, targets ARM Cortex-M IoT — middle ground. TinyOS is best for truly constrained WSN nodes; FreeRTOS for richer IoT devices.

Q9. What is RPL routing protocol?

RPL (Routing Protocol for Low-Power and Lossy Networks, RFC 6550) is the IETF standard routing protocol for 6LoWPAN. It builds a **DODAG** (Destination-Oriented Directed Acyclic Graph) rooted at the sink. Uplink (sensor to sink) follows parent links up the DAG. Downlink (Internet to sensor) uses source routing. Nodes use an **Objective Function** (e.g., ETX = Expected Transmissions, minimize energy) to select parents.

Q10. What is the need for Internet-to-WSN downlink communication?

Internet-to-WSN downlink enables: (1) **Task injection** — issuing queries like “measure temperature every 5 s for 1 hour”; (2) **Actuator control** — turning on/off actuators (valves, lights) from remote; (3) **Configuration updates** — changing sampling rate, thresholds; (4) **Firmware over-the-air (FOTA)** — updating node software remotely. Challenges: sleeping nodes, addressing, reliability over lossy links.

Q11. What is MQTT and how is it used in WSN integration?

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe protocol over TCP. A **broker** manages topics. Sensor nodes (via gateway) publish to topics (e.g., `sensor/room1/temp`); Internet applications subscribe. The broker buffers messages for offline subscribers. Advantages: very small packet overhead (2-byte header), store-and-forward for intermittent connectivity.

Q12. What is cross-layer design in WSN?

Cross-layer design violates the traditional layered (OSI) model by allowing information to be shared between non-adjacent layers (e.g., MAC, routing, application) for joint optimization. Example: LEACH combines MAC (TDMA scheduling), network layer (clustering), and application (data rate adaptation) into one cross-layer protocol. Benefit: significant energy savings over pure layered design. Drawback: reduces modularity and reusability.

Summary: WSN OS and Internet Integration

Component	Role	Key Feature
TinyOS	WSN embedded OS	Event-driven, <15 KB, non-preemptive
nesC	Programming language	Components, interfaces, split-phase
Gateway	WSN-Internet bridge	Protocol translation, buffering
6LoWPAN	Adaptation layer	IPv6 over 802.15.4, header compression
CoAP	Application protocol	REST over UDP, 4B header
RPL	Routing protocol	DODAG for 6LoWPAN mesh
MQTT	Message protocol	Pub-sub, broker, store-and-forward